

# Curso de Go

*Aula 8 - Métodos e receivers*

*Professor : Antônio Marcelo*

NÚCLEA  
ACADEMY

The logo features a stylized 'C/B' in blue and green, followed by the text 'código\_' in green, '[3]' in blue, and 'BRAZUCA' in white within a green bracket.





# Nessa Aula nos Vamos Ver

- Entender como adicionar comportamento a structs
- Diferenciar receiver por valor e por ponteiro
- Tomar decisões corretas de design
- Parte Prática



# O que é um método em Go

- **Em Go, métodos são funções associadas a tipos.**
- **Um método é uma função com um receiver.**

Exemplo de Código:

```
type Usuario struct {  
    Nome string  
}
```

```
func (u Usuario) Saudacao() {  
    fmt.Println("Olá,", u.Nome)  
}
```

Uso de Código:

```
u := Usuario{Nome: "Antonio"}  
u.Saudacao()
```



# Entendendo o Receiver

- **Receiver é o objeto que “recebe” o método.**

Exemplo de Código:

```
func (u Usuario) Saudacao()
```

- **Aqui:**
  - **u é uma cópia do objeto**
  - **Tipo é Usuario**
- **Importante: O receiver não é “this” como em outras linguagens, mas cumpre papel semelhante.**



# Receiver por Valor

- **Trabalha com cópia**
- **Não altera o original**

Exemplo de Código:

```
u := Usuario{Nome: "Antonio"}  
u.AtualizarNome("Maria")  
  
fmt.Println(u.Nome) // continua Antonio
```





# Receiver por Ponteiro

- **Trabalha com referência**
- **Altera o objeto original**

Exemplo de Código:

```
package main
```

```
import "fmt"
```

```
type Usuario struct {  
    Nome string  
}
```

```
// Receiver por ponteiro  
func (u *Usuario) AtualizarNome(novoNome string) {  
    u.Nome = novoNome  
}
```

```
func main() {
```

```
    u := Usuario{  
        Nome: "Antonio",  
    }
```

```
    u.AtualizarNome("Maria")
```

```
    fmt.Println(u.Nome)  
}
```



# Comparação Valor vs Ponteiro

- **Receiver por valor:**
  - Não altera estado
  - Mais seguro para leitura
  - Pode gerar cópia (custo)
  -
- **Receiver por ponteiro:**
  - Altera estado
  - Mais eficiente para structs grandes
  - Compartilha dados
- **Regra geral: Se o método modifica o objeto → use ponteiro**



# Quando usar ponteiros

- **Precisa modificar o estado:**

Exemplo de Código:

```
func (u *Usuario) Aniversario() {  
    u.Idade++  
}
```

- **Struct grande (performance)**
- **Evitar cópias desnecessárias**
- **Compartilhar estado entre funções**
- **Evite ponteiros quando:**
  - **Método é somente leitura simples**
  - **Struct é muito pequena**





# Métodos em Composição

- **Composição + métodos:**

Exemplo de Código:

```
type Logger struct{}
```

```
func (l Logger) Log(msg string) {  
    fmt.Println("LOG:", msg)  
}
```

```
type Servico struct {  
    Logger  
}
```

Uso:

```
s := Servico{}  
s.Log("Executando")
```

- **Explicação:**

- **Métodos são “promovidos”**
- **Reuso de comportamento**
- **Base do design em Go**



# Boas práticas de design

- **Consistência:** Se um método usa ponteiro, mantenha padrão
- **Nome do receiver curto:**

Exemplo de Código:

```
func (u Usuario) ...
```

- **Evitar lógica complexa em métodos simples**
- **Métodos devem representar comportamento do tipo**



# Padrões reais de uso

Exemplo de Código:

```
type Conta struct {  
    Saldo float64  
}
```

```
func (c *Conta) Depositar(valor float64) {  
    c.Saldo += valor  
}
```

```
func (c Conta) VerSaldo() float64 {  
    return c.Saldo  
}
```

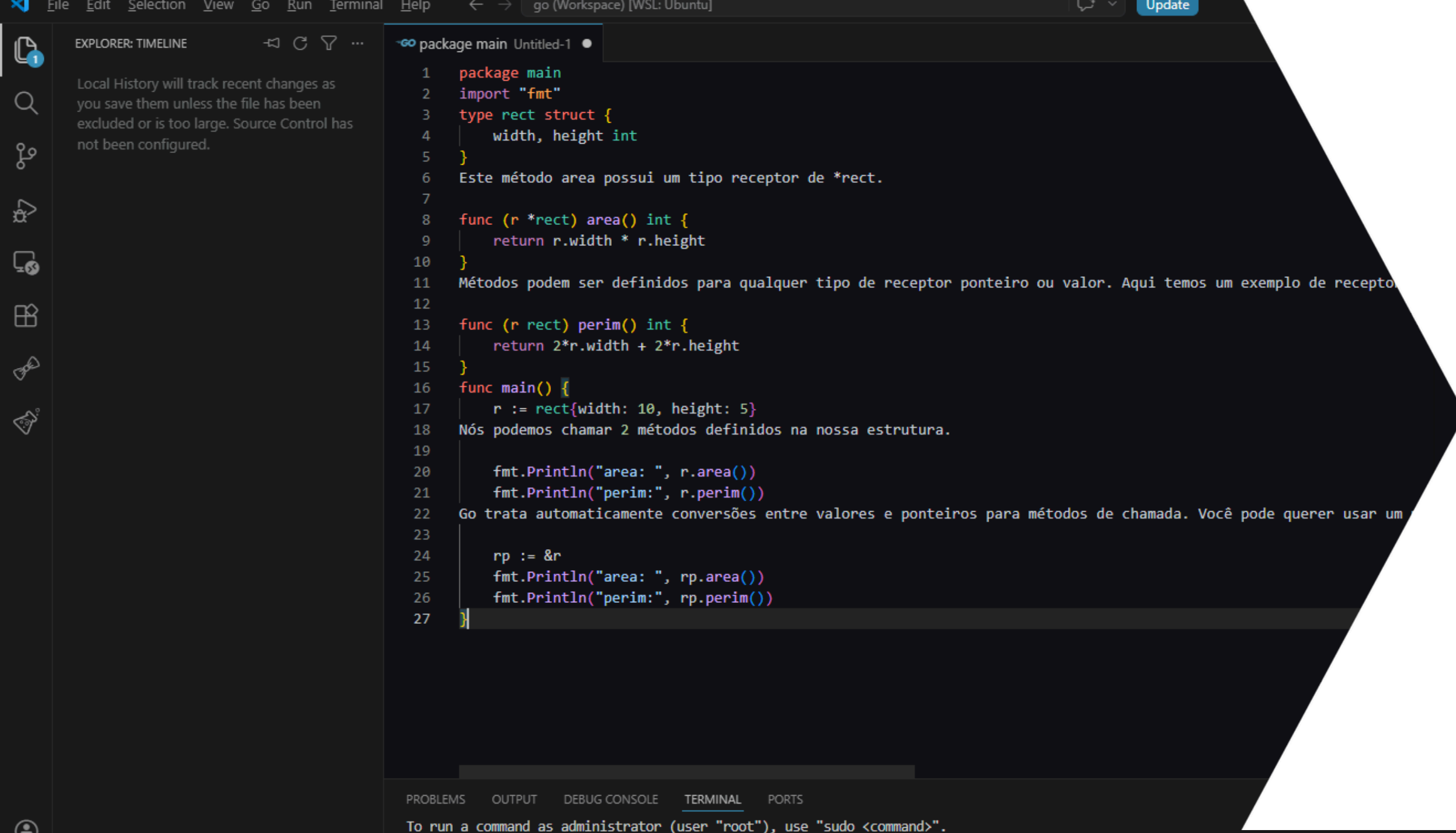
Uso

```
c := Conta{Saldo: 100}
```

```
c.Depositar(50)
```

```
fmt.Println(c.VerSaldo()) // 150
```

- **Conclusão:**
- **Métodos dão comportamento aos dados**
- **Ponteiros controlam mutabilidade**
- **Design limpo em Go depende dessas decisões**



The screenshot shows a Go IDE with a file named 'Untitled-1' in the 'package main' package. The code defines a 'rect' struct with 'width' and 'height' fields. It includes two methods: 'area()' which returns the area, and 'perim()' which returns the perimeter. The 'main' function creates a 'rect' struct with width 10 and height 5, prints its area and perimeter, and then demonstrates calling these methods on a pointer to the struct ('rp').

```
1 package main
2 import "fmt"
3 type rect struct {
4     width, height int
5 }
6 Este método area possui um tipo receptor de *rect.
7
8 func (r *rect) area() int {
9     return r.width * r.height
10 }
11 Métodos podem ser definidos para qualquer tipo de receptor ponteiro ou valor. Aqui temos um exemplo de receptor
12
13 func (r rect) perim() int {
14     return 2*r.width + 2*r.height
15 }
16 func main() {
17     r := rect{width: 10, height: 5}
18     Nós podemos chamar 2 métodos definidos na nossa estrutura.
19
20     fmt.Println("area: ", r.area())
21     fmt.Println("perim:", r.perim())
22     Go trata automaticamente conversões entre valores e ponteiros para métodos de chamada. Você pode querer usar um
23
24     rp := &r
25     fmt.Println("area: ", rp.area())
26     fmt.Println("perim:", rp.perim())
27 }
```

NÚCLEA  
ACADEMY

# Parte Prática

## Praticar Métodos e Receivers